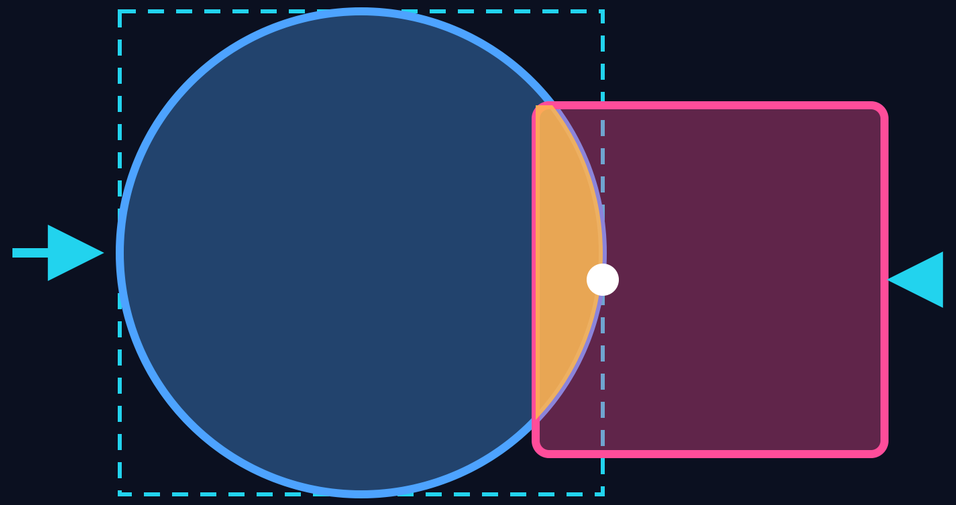
ESTRUCTURA DE DATOS II · 2026-30

Colisiones en videojuegos

Técnicas para decidir cuándo dos objetos se tocan, y cómo hacerlo rápido con estructuras de datos



IST 4031 · Universidad del Norte · Implementación práctica en Godot

María Yusti

Axel Tornay

Juan Vizcaíno

Isac Monterrosa

RECORRIDO

Índice

01 Introducción y fundamentos diap. 3–5

02 Formas de colisión diap. 6

03 Detección exacta: fase estrecha diap. 7–8

04 Detección discreta y continua diap. 9

05 Escalar con estructuras de datos diap. 10–13

06 Aplicaciones en videojuegos diap. 14

07 Implementación en Godot diap. 15–18

08 Conclusiones y referencias diap. 19–20

Qué es una colisión y por qué es un problema

El problema

Un juego avanza por cuadros. En cada uno debe decidir qué objetos se tocan y qué hacer con ese contacto.

Por qué es difícil

Hay unos 16,7 ms por cuadro a 60 FPS, y con n objetos existen n(n−1)/2 pares posibles [1].

Relación con Estructuras de Datos

Ordenar, usar hash, árboles y grafos decide cuántas pruebas se hacen. Es el mismo problema de «buscar sin revisar todo».



Origen del problema. El tiempo se simula en pasos discretos; solo vemos posiciones por cuadro, y de ahí nacen la detección, el túnel y el costo. Las técnicas combinan geometría y estructuras espaciales [1].

Fuentes: [1]

Definiciones necesarias

Colisionador (collider)
Forma simple que representa al objeto para las pruebas.

AABB
Caja alineada con los ejes X e Y; la forma más barata.

Contacto
Resultado de una prueba: normal (dirección) y profundidad (cuánto se solapan).

Fase amplia
Descarta rápido los pares que no pueden chocar.

Fase estrecha
Prueba exacta de los pares candidatos; produce el contacto.

Detección discreta
Revisa solapes solo en las posiciones de cada cuadro.

Detección continua
Revisa el recorrido entre cuadros y calcula el instante del impacto.

Túnel (tunneling)
Un objeto rápido cruza otro sin solaparse en ningún cuadro.

Fuentes: [1], [2]

El proceso: de la lista de objetos al contacto



Detectar ≠ responder

Una moneda **detecta** que entró el jugador y no bloquea nada. Una pelota **responde**: rebota. Misma detección, respuestas distintas.

Se repite en cada cuadro

Mover → detectar → responder → dibujar, unas 60 veces por segundo. Cada etapa descarta trabajo para la siguiente [2], [1].

Fuentes: [2], [1]

Del modelo detallado al colisionador

	FORMA · PRUEBA	VENTAJA	LIMITACIÓN
	**AABB / caja** 2 comparaciones por eje (2D: 4)	Muy rápida; ideal como filtro	Imprecisa si el objeto rota o es diagonal
	**Círculo / esfera** Una distancia y un radio	No depende de la rotación; muy barata	Ajusta mal a objetos alargados
	**Caja orientada (OBB)** SAT con los ejes de ambas cajas	Sigue la rotación del objeto	Más cara que la AABB
	**Cápsula (3D y 2D)** Segmento con radio: distancia a un segmento	Buena para personajes que caminan	Poco útil para formas irregulares
	**Polígono / casco convexo** SAT (o GJK) entre convexos	Se ajusta mejor al contorno	Costo crece con el número de aristas; no admite huecos
	**Malla de triángulos** Triángulo contra triángulo, con un árbol que descarta	Exacta; sirve para escenarios complejos	Muy costosa; se reserva para objetos estáticos

Hitboxes compuestos: en vez de una sola forma, un personaje puede dividirse en varias cajas (cabeza, torso, extremidades) para distinguir dónde recibe el impacto [3]. Mientras más simple la forma, más barata la prueba [1].

Fuentes: [1], [3], [4]

Fase estrecha: las pruebas simples

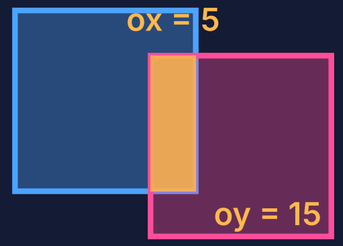
Círculo contra círculo



1. d = cB − cA
2. ¿|d| < rA + rB? 15 < 20 → sí
3. profundidad = 20 − 15 = 5

`tiempo O(1) · espacio O(1)`

Caja contra caja (AABB)



menor solape:
eje X
normal (1, 0)

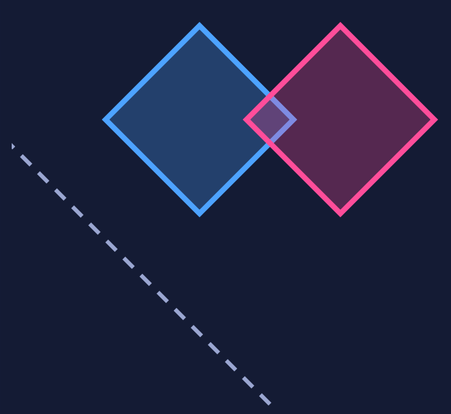
1. Proyectar en X y en Y
2. ¿Solape en los dos ejes? sí
3. El menor solape da la normal

`tiempo O(1) · espacio O(1)`

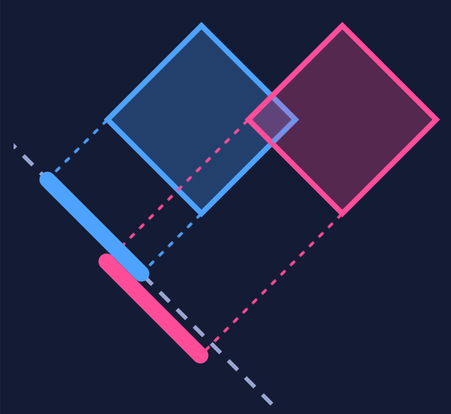
Ventaja: exactas y de costo constante. **Limitación:** solo cubren círculos y cajas alineadas; para polígonos hace falta SAT.

Fuentes: [1]

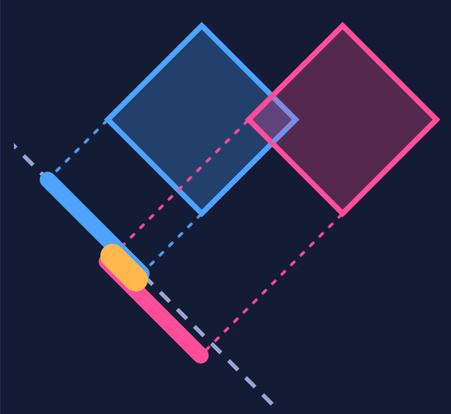
SAT: el teorema del eje separador



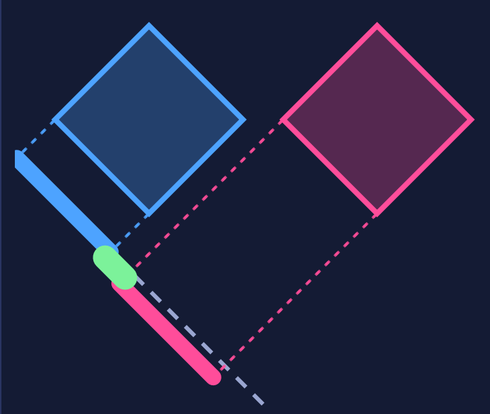
1 · Elegir un eje: la normal de una arista



2 · Proyectar las dos formas sobre el eje



3 · Solape en este eje: 3,54



4 · Un eje con hueco basta: no chocan

Si ningún eje tiene hueco → hay colisión. El menor solape da el MTV.
Limitación: solo convexos; los cóncavos se parten.

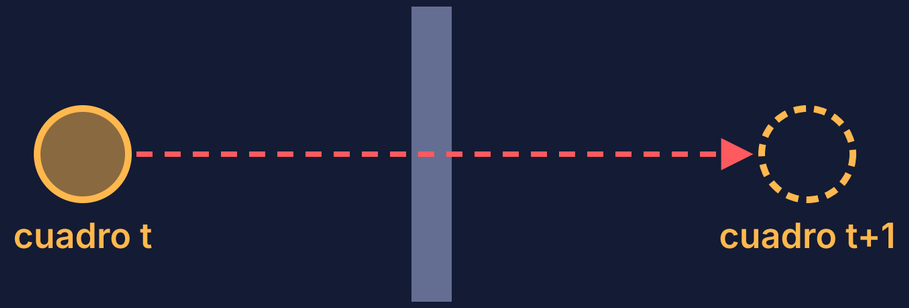
`tiempo O((a+b)²)`

a y b = número de aristas

Fuentes: [5], [1]

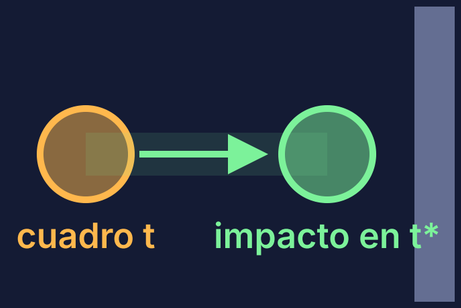
Cuándo mirar: en cada cuadro o a lo largo del trayecto

Discreta



Solo compara las posiciones de cada cuadro. **Ventaja:** barata y simple. **Limitación:** puede haber túnel con objetos rápidos o delgados.

Continua

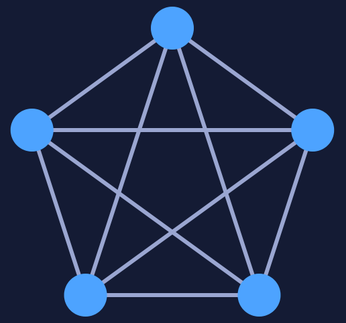


Revisa el trayecto entre cuadros y calcula cuándo ocurre el impacto. **Ventaja:** no atraviesa. **Limitación:** más costosa; se reserva para balas o vehículos.

Tiempo de impacto: si la pared está a d = 2 m y la bala va a v = 2 m/s, t* = d / v = 1 s [3]. Otras salidas: dividir el paso de física, usar paredes más gruesas o lanzar un rayo (raycast). En Godot, `RigidBody2D.continuous_cd` activa la detección continua (por defecto desactivada) [6].

Fuentes: [1], [3], [6], [2]

El problema: el costo crece con el cuadrado



5 objetos → 10 pares

Pares por revisar en cada cuadro

n = 10	45
n = 100	4.950
n = 1.000	499.500

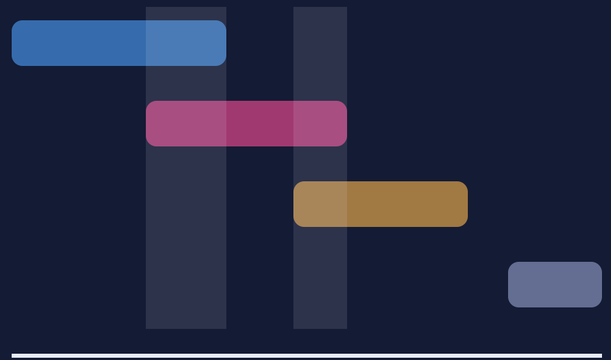
pares = $n(n - 1) / 2$ → $O(n^2)$

Presupuesto: a 60 cuadros por segundo hay unos 16,7 ms por cuadro. Probar con exactitud todos los pares no cabe.

Fuentes: [1], [7]. Complejidad: [8]

Fase amplia: evitar probar todos los pares

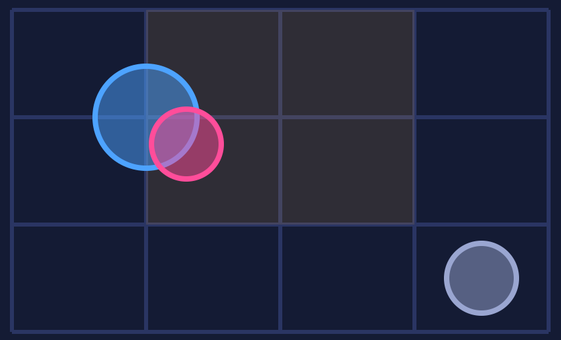
Ordenar por un eje



Solo se comparan los que se solapan en X

`O(n log n + k)`

Cuadrícula (hash)

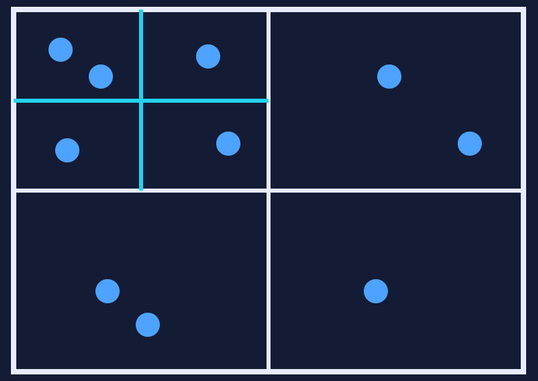


celda = f(⌊x/c⌋, ⌊y/c⌋)

Solo comparten pares los de la misma celda

`O(n + k)`

QuadTree



se divide donde hay más objetos

Cada región se subdivide en 4

`≈ O(n log n)`

k = número de pares que de verdad se solapan

Fuentes: [1], [9], [10], [7]

Lo que vimos en clase, aplicado

TEMA DE LA CLASE		EN COLISIONES
Árboles Altura = eficiencia	→	**QuadTree** Cada nodo parte el espacio en 4
Ordenamientos y búsquedas Ordenar para comparar menos	→	**Sweep and Prune** Ordena intervalos en un eje
Tablas hash Clave → posición	→	**Cuadrícula espacial** Celda = hash de la posición
Grafos Vértices, aristas, DFS	→	**Grafo de contactos** Grupos = componentes conexas
Hilos Repartir un problema grande	→	**Fases en paralelo** Idea; no implementada aquí

Fuentes: material de clase [11], [8], [12], [13], [14]; [1]

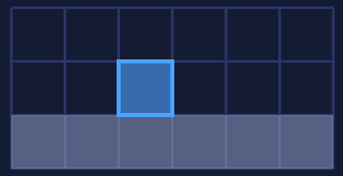
Comparación: ventajas y limitaciones

TÉCNICA	VENTAJA	LIMITACIÓN	ENCAJA EN
AABB / círculo	O(1); muy barata	Solo ajuste aproximado	Plataformas 2D, filtros
SAT (convexos)	Exacta; da normal y profundidad	$O((a+b)^2)$; no sirve para cóncavos	Obstáculos poligonales
Sweep and Prune	Aprovecha que los objetos se mueven poco entre cuadros	Se degrada si muchos se solapan en el eje	Muchos objetos en movimiento
Cuadrícula (hash)	Consulta casi O(1) por celda	Mal con tamaños muy distintos o objetos agrupados	Mundos de bloques, muchas balas
QuadTree / Octree	Se adapta a zonas densas y vacías	Hay que reconstruirlo o actualizarlo si los objetos se mueven	Niveles grandes y desiguales
Continua	No hay túnel	Más cara	Proyectiles rápidos

No hay técnica universal: los motores combinan una fase amplia barata con una fase estrecha exacta y eligen según el tipo de juego [1], [7].

Fuentes: [1], [9], [10], [7]

Cuatro aplicaciones reales de las técnicas



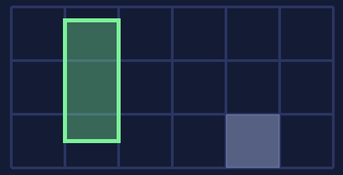
Plataformas 2D

Forma: caja AABB del personaje
Técnica: cuadrícula de baldosas: solo se prueban las baldosas vecinas
Respuesta: frenar o deslizar contra el suelo y las paredes



Disparos en 3D

Forma: varias cajas por parte del cuerpo
Técnica: rayo (raycast) o detección continua para balas rápidas
Respuesta: registrar el impacto y la zona herida



Mundos de bloques

Forma: caja de cada entidad
Técnica: cuadrícula entera: solo los bloques que la caja ocupa
Respuesta: impedir atravesar bloques sólidos

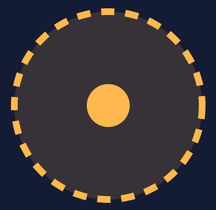


Billar y física

Forma: círculos o esferas
Técnica: fase estrecha de distancias y respuesta por rebote
Respuesta: rebote según la normal del contacto

Fuentes: [1], [3], [2], [15]

Cuatro tipos de cuerpo



Area2D

Detecta y avisa con señales. No bloquea.

`en el proyecto: Moneda`



StaticBody2D

No se mueve. Otros chocan con él.

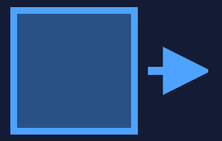
`en el proyecto: Paredes, Obstaculo`



RigidBody2D

El motor calcula la respuesta.

`en el proyecto: Pelota`



CharacterBody2D

Tú programas el movimiento y la respuesta.

`en el proyecto: Jugador`

Cada uno lleva un `CollisionShape2D` con su forma.

Por qué importa: la elección del cuerpo decide quién calcula la respuesta, el motor o el programador.

Fuentes: [2], [15], [16], [6]

Capas y máscaras filtran quién choca con quién

¿Con qué capas choca cada uno? (máscara)

	capa 1 paredes	capa 2 jugador	capa 3 pelotas	capa 4 monedas
Jugador	✓		✓	
Pelota	✓	✓	✓	
Moneda		✓		

collision_layer
en qué capa **está** el objeto

collision_mask
qué capas **escanea**

La moneda solo escanea al jugador: las pelotas la atraviesan.

Ventaja: se filtran pares sin escribir código y se evitan pruebas innecesarias. **Limitación:** hay 32 capas; si la máscara está mal configurada, el objeto simplemente no choca ni avisa.

Ejemplo del proyecto: al recoger una moneda solo importa el jugador. Con la máscara en la capa 2, las tres pelotas pasan sobre ella sin activar la señal `body_entered`.

Fuentes: [2]. Proyecto: project.godot, escenas/*.tscn

Escena y código del proyecto

Escena laboratorio.tscn

```
Laboratorio  (Node2D)
├─ Paredes  (StaticBody2D)
├─ Obstaculo  (StaticBody2D)
├─ Jugador  (CharacterBody2D)
├─ Pelotas
│   ├─ Pelota1  (RigidBody2D)
│   ├─ Pelota2
│   └─ Pelota3
├─ Monedas
│   ├─ Moneda1  (Area2D)
│   ├─ Moneda2
│   └─ Moneda3
├─ Superposicion  (Node2D)
└─ Interfaz  (CanvasLayer)
```

Dos formas de reaccionar

```gdscript
# jugador.gd  — respuesta programada
velocity = direccion * velocidad
move_and_slide()
if get_slide_collision_count() > 0:
    var info = get_slide_collision(0)
    choque.emit(info.get_collider(), info.get_normal())

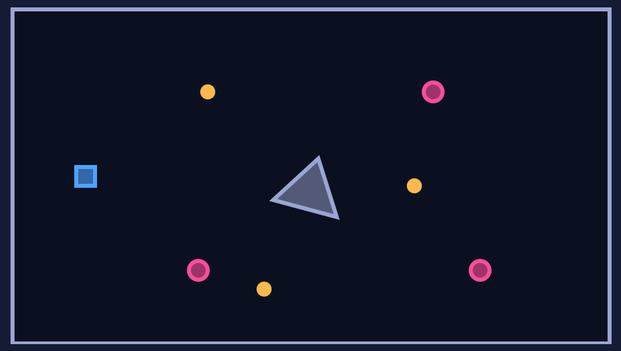
# moneda.gd  — solo detección
func _al_entrar_cuerpo(cuerpo):
    if cuerpo.is_in_group("jugador"):
        recogida.emit(self)
        queue_free()
```

Relación con la teoría: `get_slide_collision` entrega la normal del contacto; la moneda solo detecta; el rebote lo calcula el motor.

Fuentes: [15], [16]. Proyecto: scripts/jugador.gd, scripts/moneda.gd

Demostración y origen del código

Escena del laboratorio



Esquema con las posiciones reales de laboratorio.tscn (no es una captura).

■ jugador · ● pelotas · ● monedas · ▲ obstáculo

Casos de la demostración en vivo

1. **Mover al jugador contra una pared**
   Se detiene y se desliza; sube el contador de choques
2. **Dejar que una pelota golpee al jugador**
   La pelota rebota; sube el contador de rebotes
3. **Tocar una moneda**
   La moneda desaparece; sube el contador de monedas
4. **Pulsar Espacio**
   Se dibujan las cajas AABB y las líneas de contacto
5. **Ejecutar las pruebas manuales**
   Imprime «Pruebas: 17 | fallos: 0» si todo es correcto

Origen del código: escrito por el grupo con asistencia de IA a partir de [1], [5] y de la documentación oficial de Godot [2], [16], [15], [6]. Las pruebas manuales se tradujeron a Python para comprobar sus resultados.

Fuentes: [1], [5], [2], [15]. Proyecto: escenas/laboratorio.tscn, scripts/pruebas_manuales.gd

Conclusiones

1. Detectar y responder son pasos distintos: el contacto (normal y profundidad) conecta ambos.
2. Se filtra primero con formas y estructuras baratas y luego se mide con exactitud.
3. Discreta o continua: depende de la velocidad y el tamaño del objeto.
4. Las estructuras de datos de la clase (árboles, orden, hash, grafos) reducen el costo $O(n^2)$.
5. No hay técnica universal; cada juego equilibra precisión y rendimiento.

Referencias (IEEE)

[1] C. Ericson, *Real-Time Collision Detection*. San Francisco, CA, EE. UU.: Morgan Kaufmann, 2004.

[2] Godot Engine contributors, «Physics introduction», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/physics_introduction.html

[3] «How Collisions Work in Games», video explicativo, 5 min 53 s. Archivo proporcionado por el grupo; autor y URL no verificados.

[4] Godot Engine contributors, «Collision shapes (2D)», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/collision_shapes_2d.html

[5] Kansas State University, «Separating axis theorem», *CIS 580 Textbook*, s. f. [En línea]. Disponible: https://textbooks.cs.ksu.edu/cis580/04-collisions/04-separating-axis-theorem/

[6] Godot Engine contributors, «RigidBody2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/classes/class_rigidbody2d.html

[7] M. Lysenko, «Collision detection (part 1): Overview», *0 FPS*, ene. 2015. [En línea]. Disponible: https://0fps.net/2015/01/07/collision-detection-part-1/

[8] E. D. Angulo Madrid, «Ordenamientos y búsquedas», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[9] B. Mirtich, «Efficient algorithms for two-phase collision detection», Mitsubishi Electric Research Laboratories, Informe técnico TR-97-23, dic. 1997. [En línea]. Disponible: https://merl.com/publications/docs/TR97-23.pdf

[10] R. A. Finkel y J. L. Bentley, «Quad trees: A data structure for retrieval on composite keys», *Acta Informatica*, vol. 4, n.º 1, pp. 1–9, 1974, doi: 10.1007/BF00288933.

[11] E. D. Angulo Madrid, «Árboles», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[12] E. D. Angulo Madrid, «Búsqueda y organización de la información», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[13] E. D. Angulo Madrid, «Grafos», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[14] E. D. Angulo Madrid, «Hilos y sockets», diapositivas de clase, Estructura de Datos II, Universidad del Norte, s. f.

[15] Godot Engine contributors, «Using CharacterBody2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/using_character_body_2d.html

[16] Godot Engine contributors, «Using Area2D», *Godot Docs*, s. f. [En línea]. Disponible: https://docs.godotengine.org/en/stable/tutorials/physics/using_area_2d.html